Algorithm

An algorithm is a set of steps of operations to solve a problem performing calculation, data processing, and automated reasoning tasks.

An algorithm is an efficient method that can be expressed within a finite amount of time and space.

The important aspects of algorithm design include creating an efficient algorithm to solve a problem in an efficient way using minimum time and space.

To solve a problem, different approaches can be followed. Some of them can be efficient with respect to time consumption, whereas other approaches may be memory efficient.

Properties of an algorithm

TO EVALUATE AN ALGORITHM WE HAVE TO SATISFY THE FOLLOWING CRITERIA:

1. **INPUT:** The Algorithm should be given zero or more input.
2. **OUTPUT:** At least one quantity is produced. For each input the algorithm produced value from a specific task.
3. **DEFINITENESS:** Each instruction is clear and unambiguous.
4. **FINITENESS:** If we trace out the instructions of an algorithm, then for all cases, the algorithm terminates after a finite number of steps.
5. **EFFECTIVENESS:** Every instruction must be very basic so that it can be carried out, in principle, by a person using only pencil & paper.

PERFORMANCE ANALYSIS

Performance Analysis: An algorithm that is said to be efficient and fast if it takes less time to execute and consumes less memory space at run time is called Performance Analysis.

1. **SPACE COMPLEXITY:** The space complexity of an algorithm is the amount of Memory Space required by an algorithm during course of execution is called space complexity .There are three types of space

a)Instruction space :executable program

b)Data space: Required to store all the constant and variable data space.

c)Environment: It is required to store environment information needed to resume the suspended space.

Now there are two types of space complexity

a) Constant space complexity

b) Linear(variable)space complexity

2.Linear space complexity: The space needed for the algorithm is based on size.

- Size of the variable ‘n’ = 1 word
- Array of a values = n word
- Loop variable = 1 word
- Sum variable = 1 word

Example:

```
int sum(int A[],int n)
{                          n
int sum=0,i;               1
```

```
for (i=0;i<n;i++)        1
Sum=sum+A[i];            1
Return sum;
}
```

Ans : 1+n+1+1 = n+3 words

2. TIME COMPLEXITY: The time complexity of an algorithm is the total amount of time required by an algorithm to complete its execution.

1.Constant time complexity : If a program required fixed amount of time for all input values is called Constant time complexity .

Example :
```
int sum(int a , int b)
{
return a+b;
}
```

2.Linear time complexity: If the input values are increased then the time complexity will change.

- comments = 0 step
- Assignment statement= 1 step
- condition statement= 1 step
- loop condition for n times = n+1 steps
- body of the loop = n steps

Example :
```
int sum(int A[],int n)
{
int sum=0,i;
for (i=0;i<n;i++)
sum=sum+A[i];
return sum;
```

cost	repetation	total
1	1	1
1+1+1	1+(n+1)+n	2n+2
2	n	2n
1	1	1
		4n+4

Analysis of an Algorithm

The goal of analysis of an algorithm is to compare algorithms in running time and also Memory management.

Running time of an algorithm depends on how long it takes a computer to run the lines of code of the algorithm.

Running time of an algorithm depends on

1.Speed of computer

2.Programming language

3.Compiler and translator

Examples: binary search, linear search

ASYMPTOTIC ANALYSIS:

Expressing the complexity in terms of its relationship to known function. This type analysis is called asymptotic analysis.

The main idea of Asymptotic analysis is to have a measure of efficiency of an algorithm , that doesn't depends on

1.Machine constants.

2.Doesn't require an algorithm to be implemented.

3.Time taken by program to be prepared.

There are 3 asymptotic notations that are mostly used to represent time complexity of algorithms.

1.Big oh (O)notation

2.Big omega (Ω) notation

3.Theta(Θ) notation

1.Big oh (O)notation : Asymptotic "less than"(slower rate).This notation mainly represent upper bound of algorithm run time.

Big oh (O)notation is useful to calculate maximum amount of time of execution.

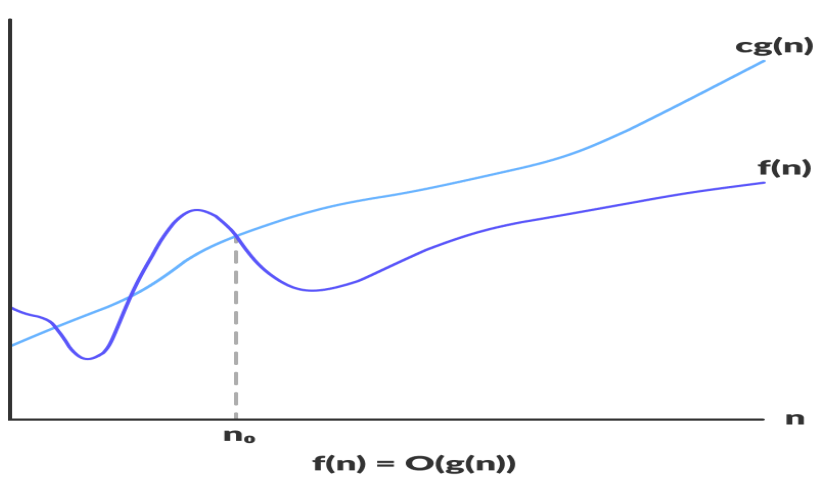
By using Big-oh notation we have to calculate worst case time complexity.

Formula : $f(n)<=c\ g(n)$ $n>=n_0$, $c>0$, $n_0>=1$

Definition: Let f(n) ,g(n) be two non negative (positive) function

now the f(n)=O(g(n)) if there exist two positive constant c,n0 such that

f(n)<= c.g(n) for all value of n>0 & c>0



Example : f(n)=3n +2 & g(n)= n

Formula : $f(n)<=c\ g(n)$ $n>=n_0$, $c>0$, $n_0>=1$

f(n)=3n+2 & g(n)=n

Now 3n+2<=c.n

3n+2<=4.n

Put the value of n =1

5<=4 false

N=2 8<=8 true now n0>2 For all value of n>=2 & c=4

now f(n)<= c.g(n)

3n+2<=4n for all value of n>=2

Above condition is satisfied this notation takes maximum amount of time to execute .so that it is called worst case complexity.

Ω-Omega notation : Asymptotic "greater than"(faster rate).

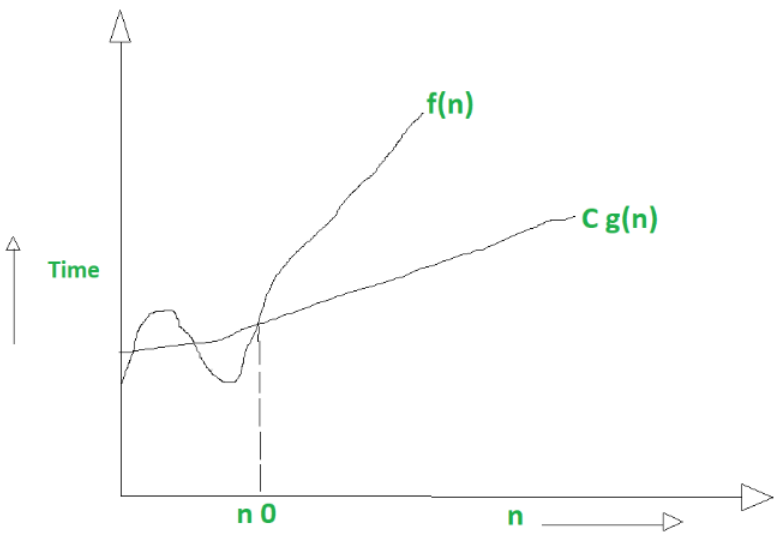
It represents the lower bound of algorithm run time.

By using Big Omega notation we can calculate minimum amount of time. We can say that it is best case time complexity.

Formula : f(n)>=c g(n) $n>=n_0$, $c>0$, $n_0>=1$

where c is constant, n is function

- Lower bound
- Best case



Example : f(n)=3n +2

Formula : f(n)>=c g(n) $n>=n_0$, $c>0$, $n_0>=1$

f(n)=3n+2

3n+2>=1*n, c=1 put the value of n=1

n=1 5>=1 true n0>=1 for all value of n

It means that f(n)= Ω g(n).

Theta (Θ) notation : Asymptotic "Equality"(same rate).

It represents the average bond of algorithm running time.

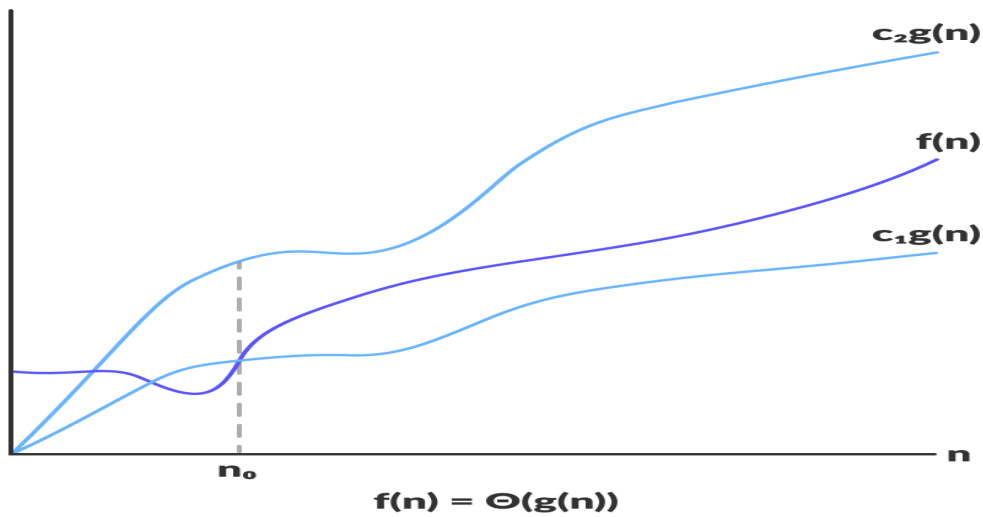
By using theta notation we can calculate the average amount of time.

So it is called average case time complexity of the algorithm.

Formula : $c_1\ g(n) <= f(n) <= c_2\ g(n)$

where c1, c2 is constants, n is function

- Average bound



Example : f(n)=3n+2

Formula : $c_1\ g(n) <= f(n) <= c_2\ g(n)$

f(n)=3n+2

1*n<=3n+2<=4*n now put the value of n=1 we get 1<=5<=4 false

n=2 we get 2<=8<=8 true

n=3 we get 3<=11<=12 true

Now all values of n>=2 are satisfied.

SEARCHING ALGORITHMS

LINEAR SEARCH

Definition

Linear Search checks each element **one by one** until the target element is found or the list ends.

Algorithm (Linear Search)

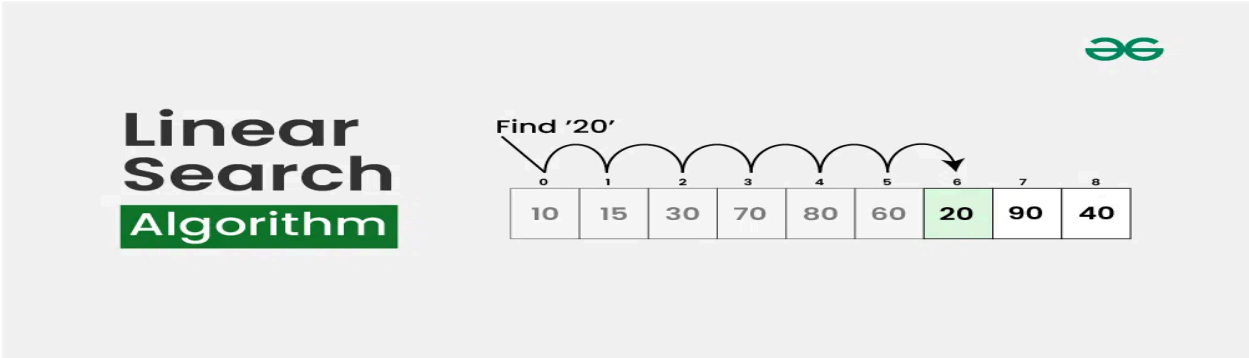
1. Start from the first element
2. Compare the key with current element
3. If match found → return index

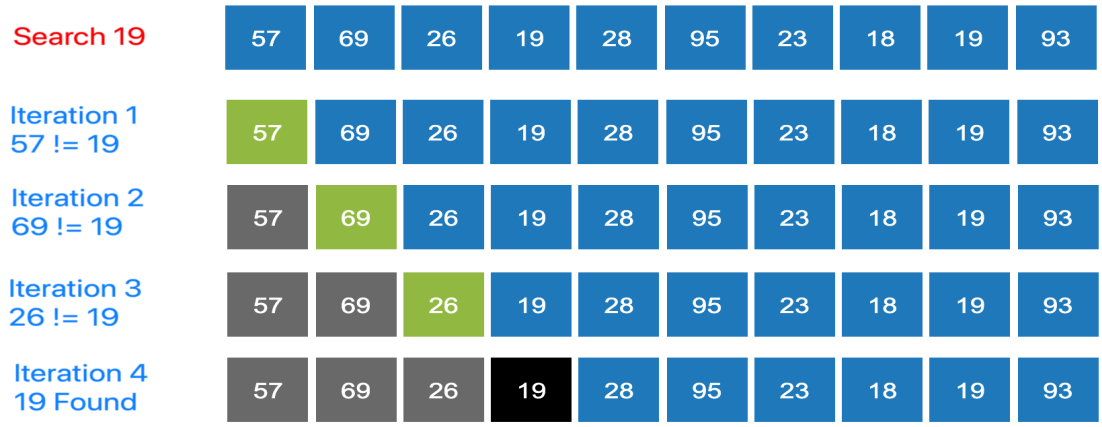
4. Else move to next element
5. If end reached → element not found

Step-by-Step Tracing

Array: `[10, 25, 30, 45, 60]`
Key: `45`

```
Step 1: Compare 10 ❌
Step 2: Compare 25 ❌
Step 3: Compare 30 ❌
Step 4: Compare 45 ✅ FOUND
```





Python Code

```python
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

arr = [10, 25, 30, 45, 60]
key = 45
print(linear_search(arr, key))
```

Time Complexity

- Best: **O(1)**
- Worst: **O(n)**

BINARY SEARCH (Sorted List)

Definition

Binary Search repeatedly divides the sorted list into halves to locate the key.

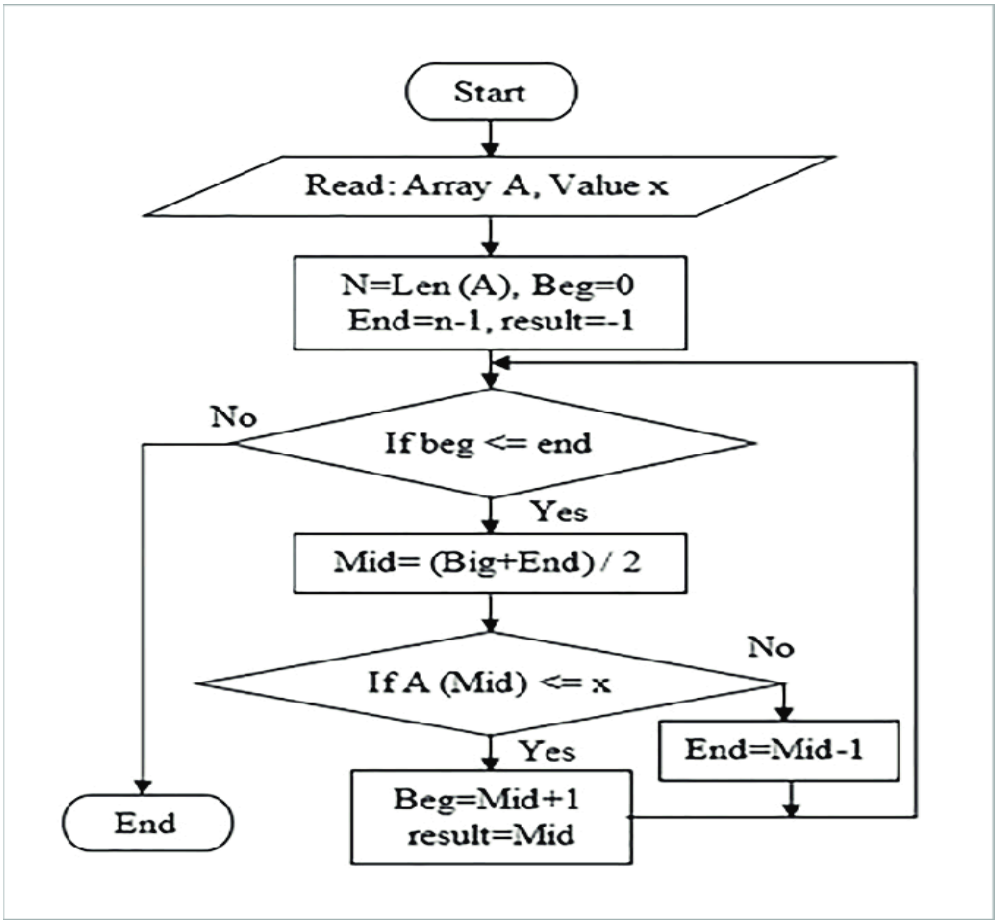
Algorithm

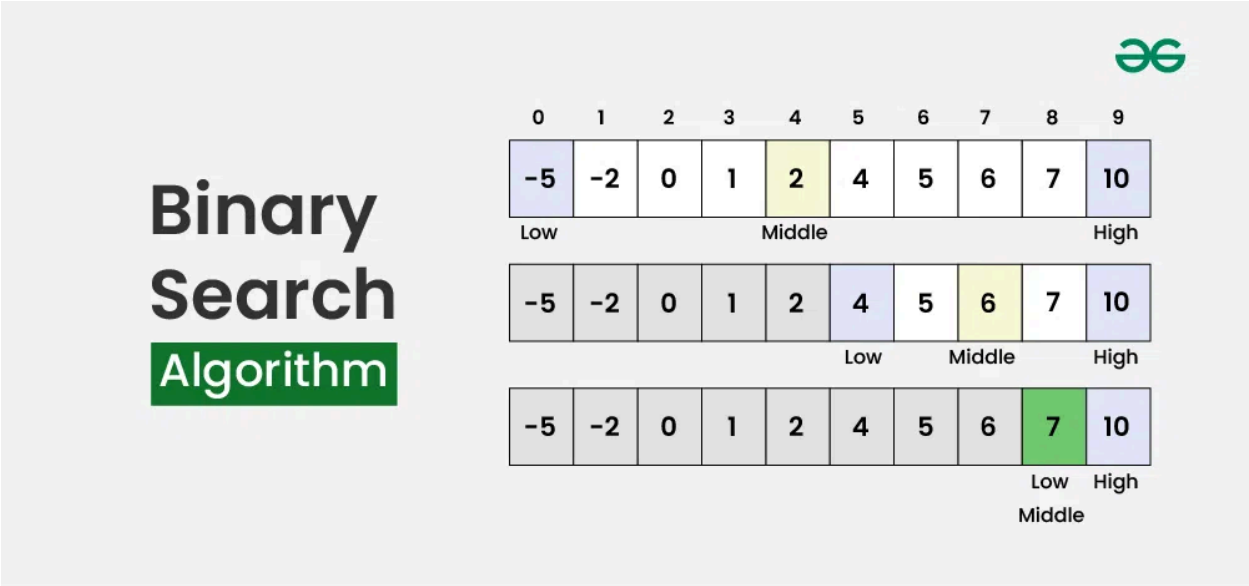
1. Set `low = 0`, `high = n-1`
2. Find `mid = (low + high) // 2`
3. If `arr[mid] == key` → found
4. If `key < arr[mid]` → search left half
5. Else → search right half
6. Repeat until found or range empty

Step-by-Step Tracing

Array: `[10, 20, 30, 40, 50]`
Key: `40`

```
mid = 30 → key > 30 → right
mid = 40 → FOUND
```





Python Code

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
```

```
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return -1
```

Time Complexity

- Best: **O(1)**
- Worst: **O(log n)**

SORTING ALGORITHMS

BUBBLE SORT

Definition

Bubble Sort repeatedly swaps adjacent elements if they are in the wrong order.

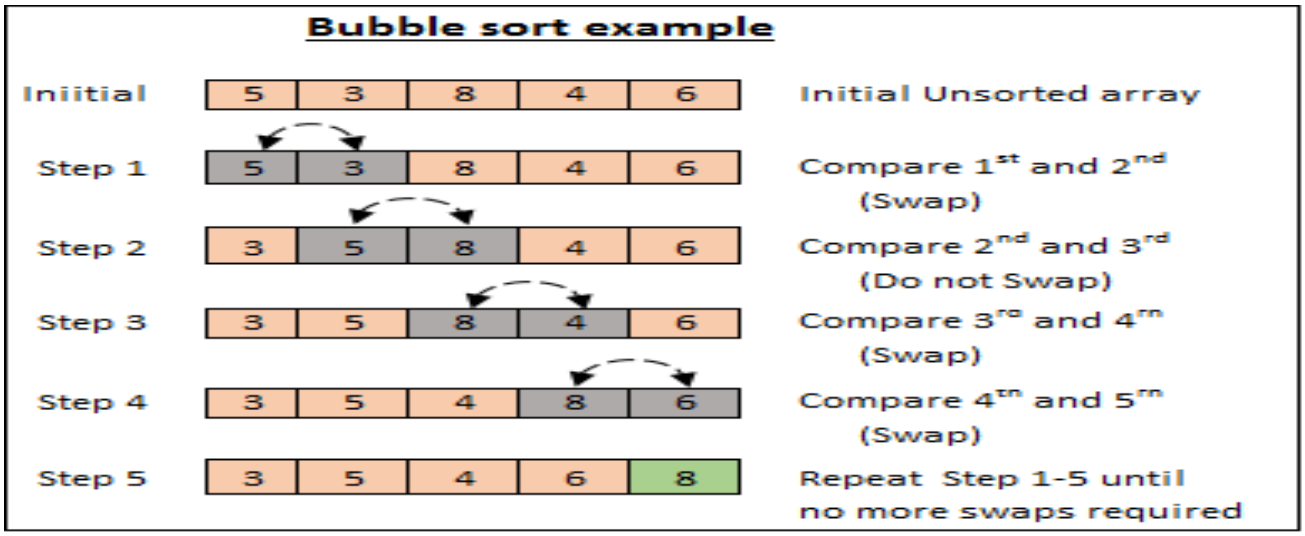
Algorithm

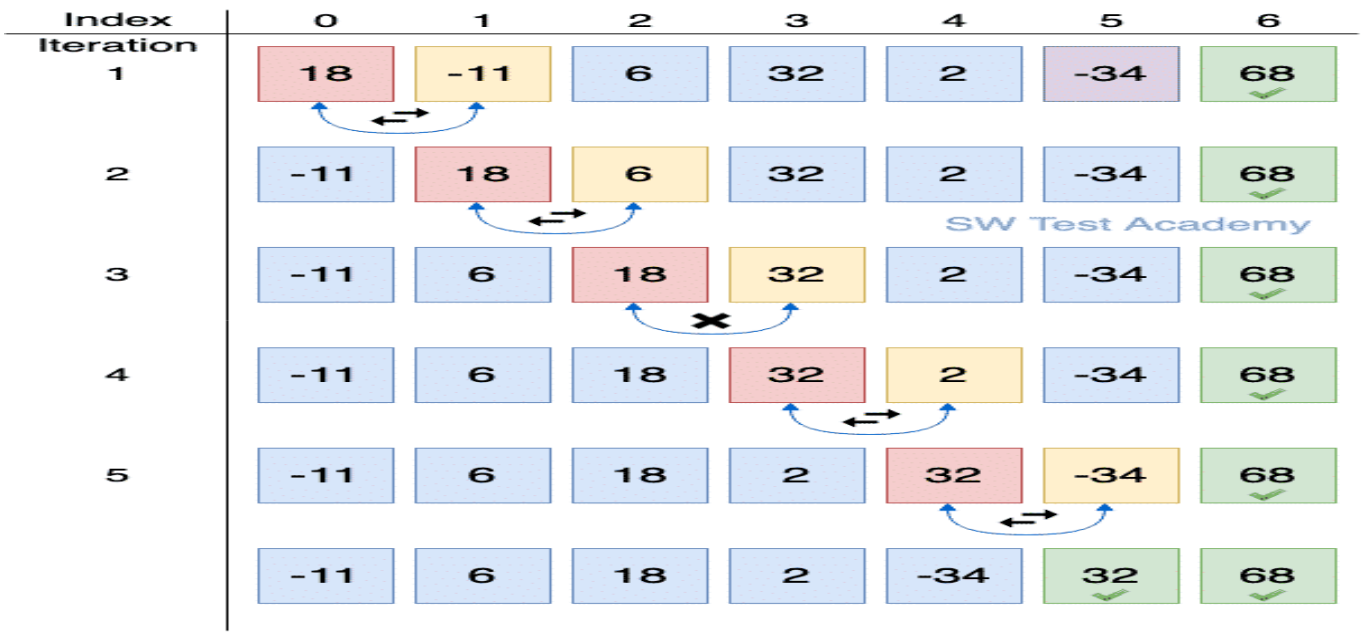
1. Compare adjacent elements
2. Swap if first > second
3. Repeat passes until no swaps

Step-by-Step Tracing

Array: `[5, 1, 4, 2]`

```
Pass 1: [1, 4, 2, 5]
Pass 2: [1, 2, 4, 5]
Sorted
```





BUBBLE SORT

Definition

Bubble Sort repeatedly swaps adjacent elements if they are in the wrong order.

Algorithm

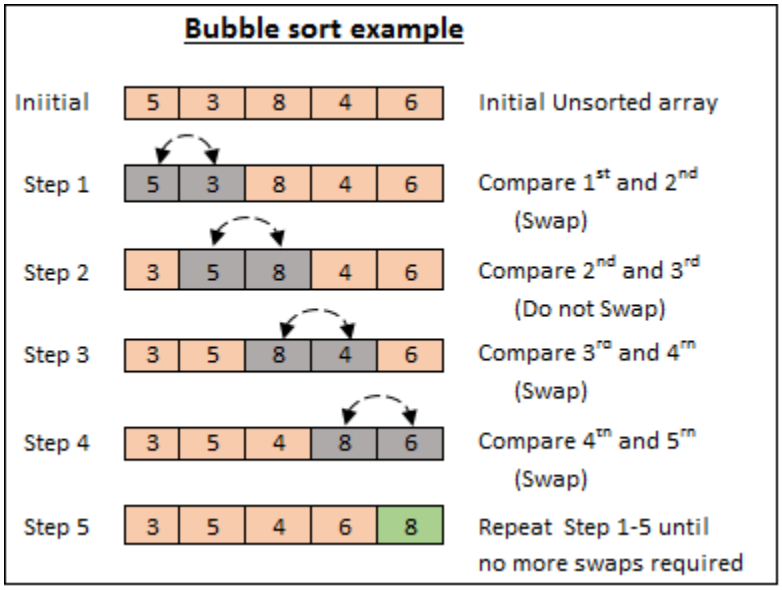
1. Compare adjacent elements
2. Swap if first > second
3. Repeat passes until no swaps

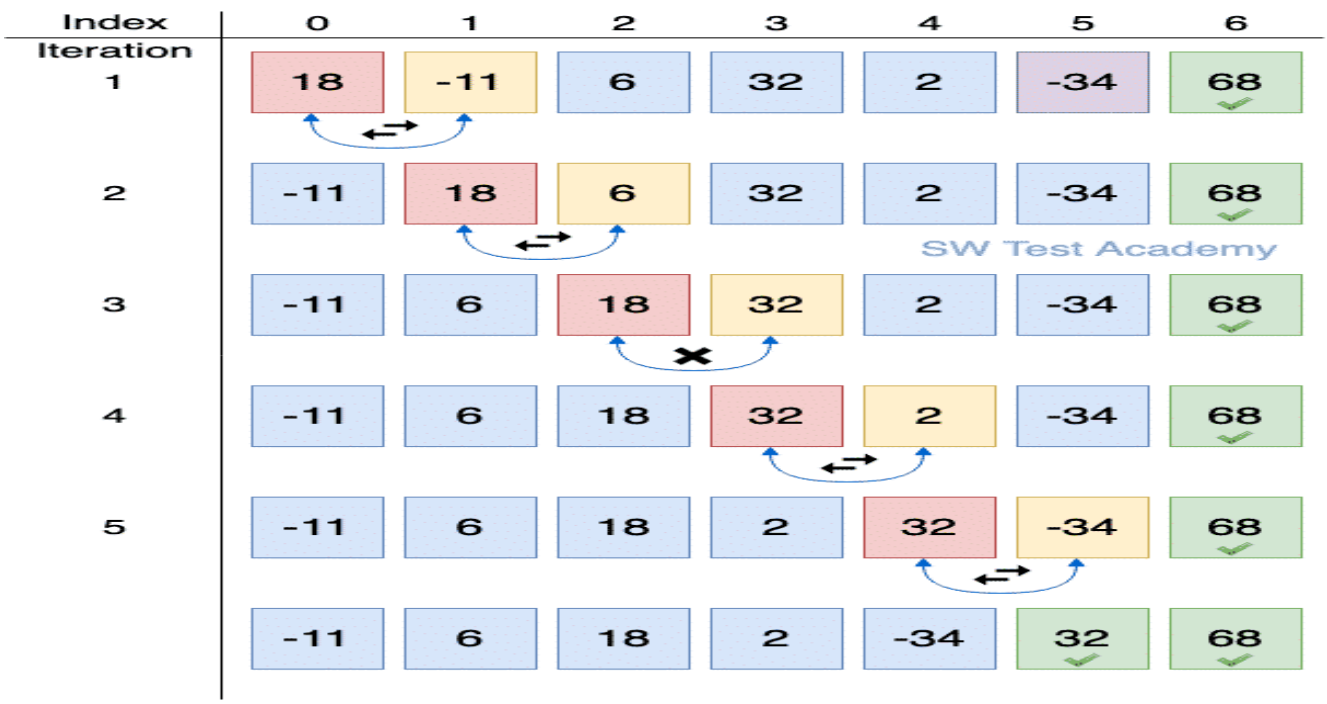
Step-by-Step Tracing

Array: `[5, 1, 4, 2]`

```
Pass 1: [1, 4, 2, 5]
Pass 2: [1, 2, 4, 5]
Sorted
```

Pictographical Representation





Python Code

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
```

```
arr[j], arr[j+1] = arr[j+1], arr[j]
```

Complexity

- Worst: **O(n²)**

INSERTION SORT

Definition

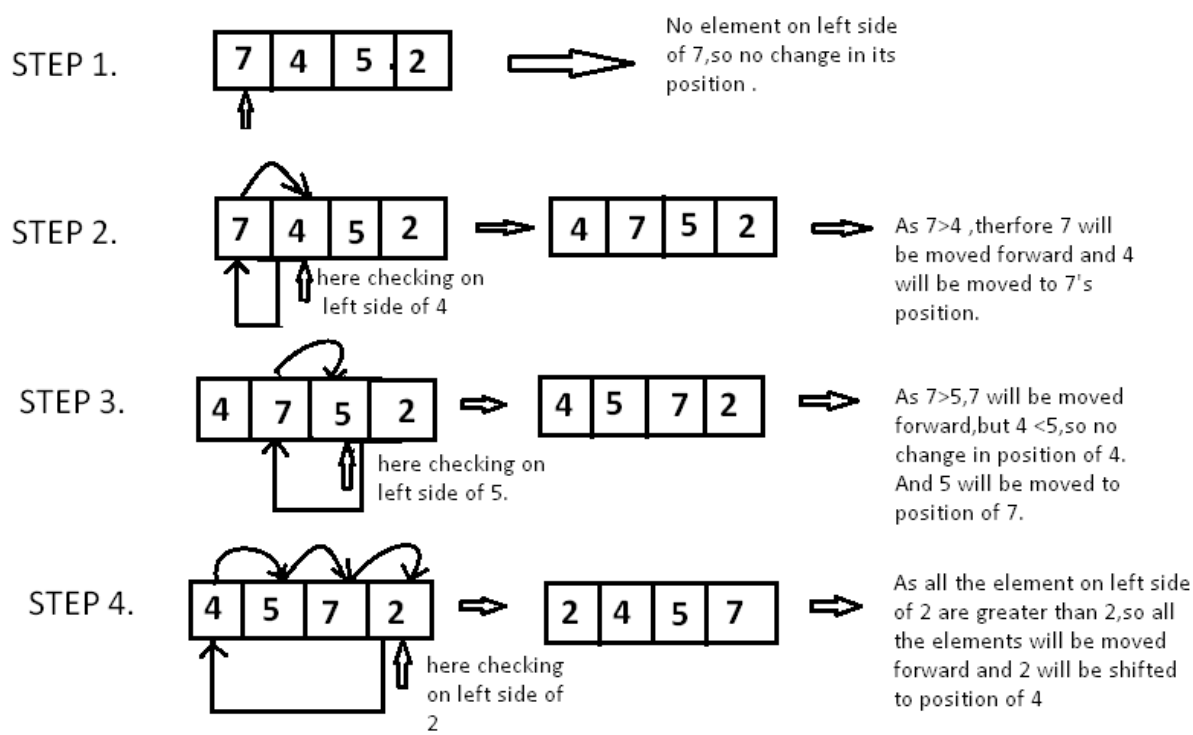
Insertion Sort builds the sorted array one element at a time.

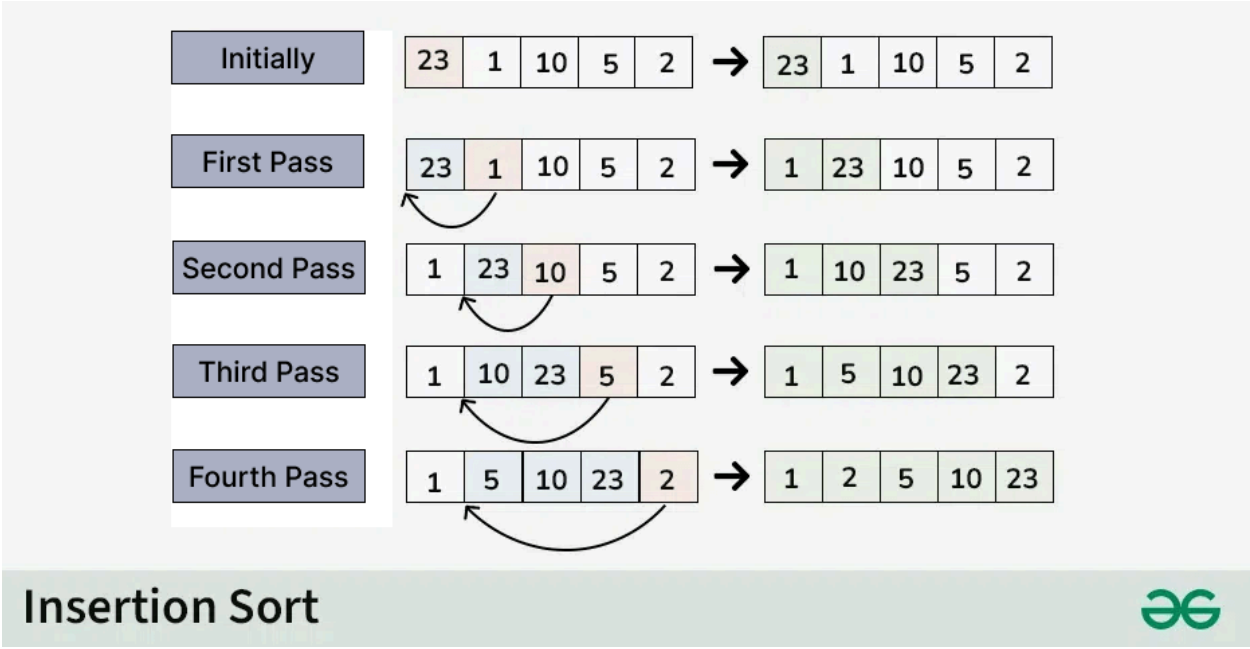
Tracing Example

Array: `[8, 3, 5, 2]`

```
Insert 3 → [3,8]
Insert 5 → [3,5,8]
Insert 2 → [2,3,5,8]
```

Pictographical Representation





Python Code

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
```

SELECTION SORT

Definition

Selection Sort selects the smallest element and places it at the correct position.

Tracing

```
Find min → swap → repeat
```

Pictographical Representation

Python Code

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
```

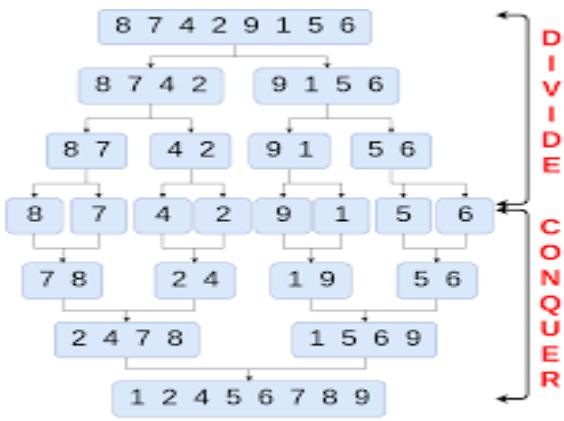
```
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

MERGE SORT (Divide & Conquer)

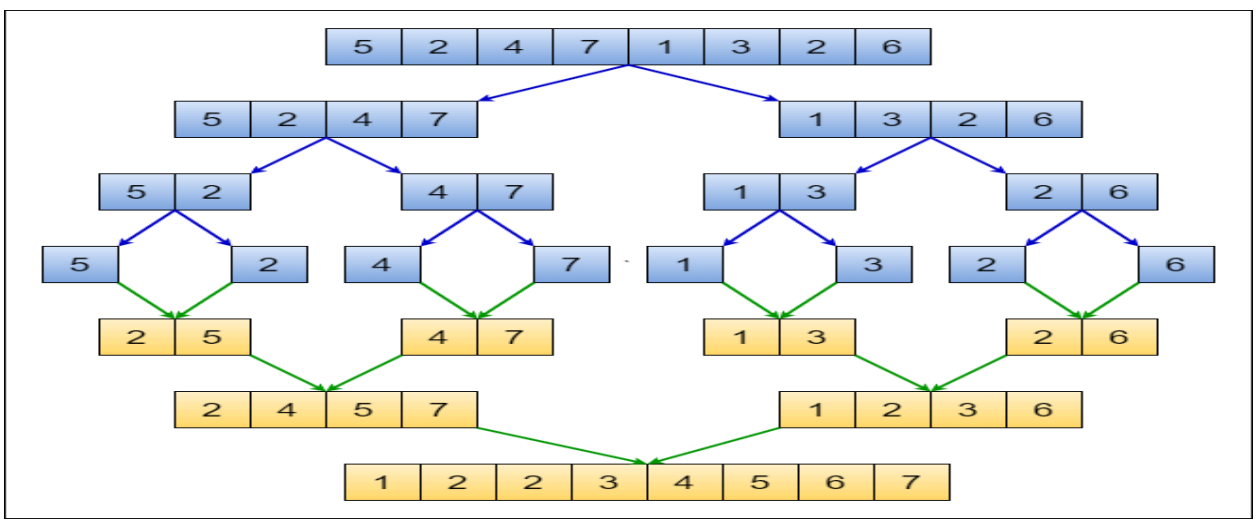
Definition

Merge Sort divides arrays into halves, sorts them, and merges.

Pictographical Representation



Merge Sort



Python Code

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr)//2
        L = arr[:mid]
        R = arr[mid:]
        merge_sort(L)
```

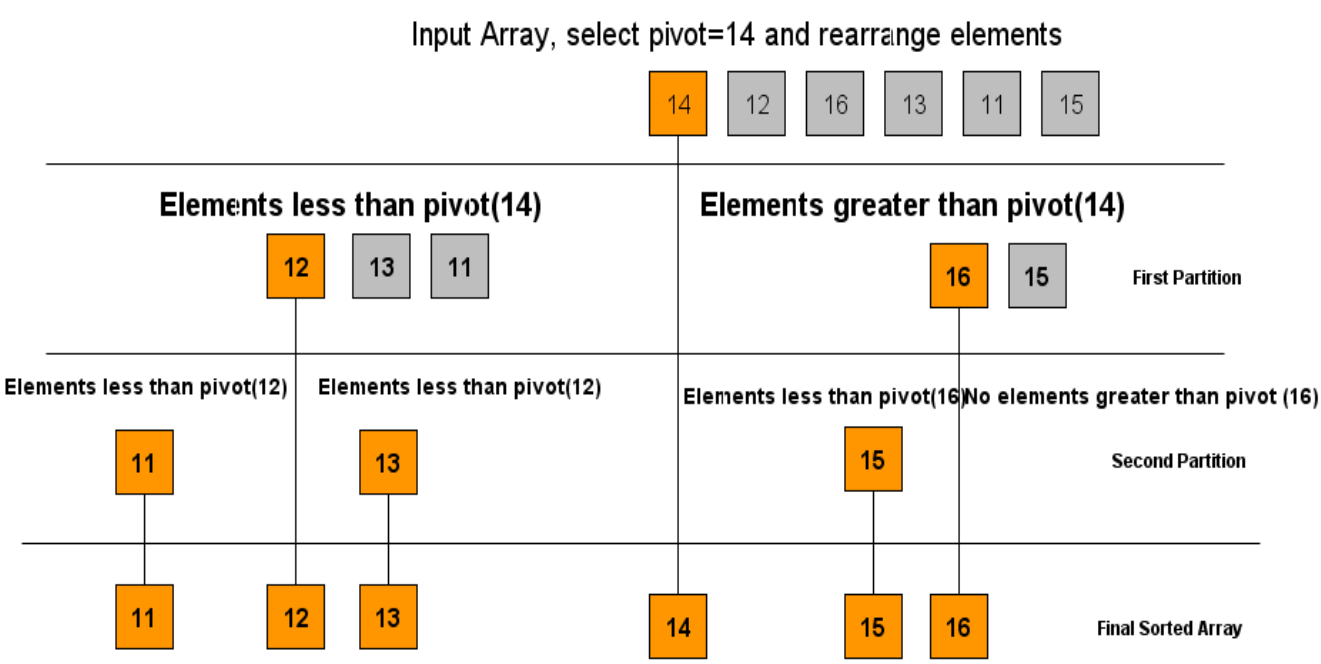
```
merge_sort(R)
i = j = k = 0
while i < len(L) and j < len(R):
    if L[i] < R[j]:
        arr[k] = L[i]
        i += 1
    else:
        arr[k] = R[j]
        j += 1
    k += 1
while i < len(L):
    arr[k] = L[i]
    i += 1
    k += 1
while j < len(R):
    arr[k] = R[j]
    j += 1
    k += 1
```

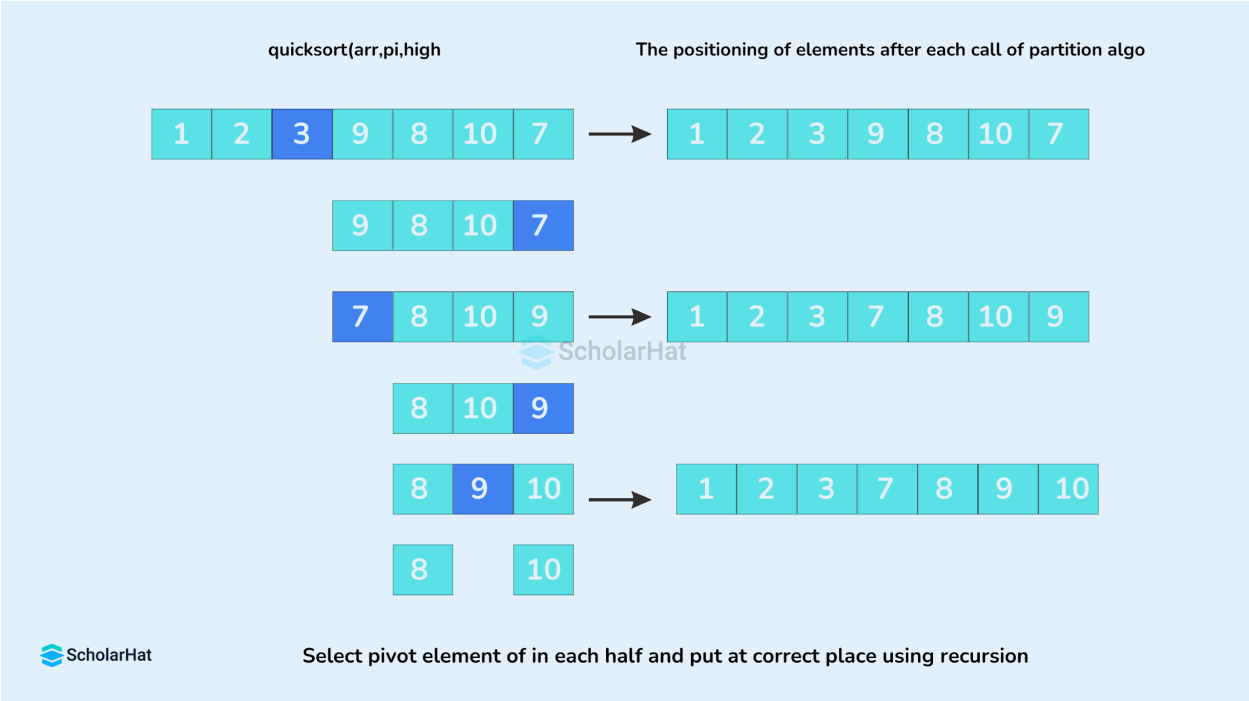
QUICK SORT

Definition

Quick Sort selects a pivot, partitions array, and sorts recursively.

Pictographical Representation





Python Code

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
```

COMPARISON TABLE

Algorithm	Best	Worst	Stable
Bubble	O(n)	O(n²)	Yes
Insertion	O(n)	O(n²)	Yes
Selection	O(n²)	O(n²)	No
Merge	O(n log n)	O(n log n)	Yes
Quick	O(n log n)	O(n²)	No

AN EXPONENTIAL ALGORITHM

RECURSIVE FIBONACCI

What is an Exponential Algorithm?

An **exponential algorithm** is one whose time complexity grows exponentially with input size, usually **$O(2^n)$**.
Such algorithms become **very slow** as `n` increases.

Recursive Fibonacci is a **classic example** of an exponential-time algorithm.

Fibonacci Series

The Fibonacci series is defined as:

```
F(0) = 0
F(1) = 1
F(n) = F(n-1) + F(n-2)   for n ≥ 2
```

Example Series:

```
0, 1, 1, 2, 3, 5, 8, 13, 21, ...
```

Why is Recursive Fibonacci Exponential?

- Each function call makes **two recursive calls**
- Many values are **recomputed repeatedly**
- Number of calls grows exponentially

Algorithm: Recursive Fibonacci

Algorithm Fibonacci(n)

1. If `n == 0`, return 0
2. If `n == 1`, return 1
3. Else
    - return `Fibonacci(n-1) + Fibonacci(n-2)`

Pseudocode

```
FIBONACCI(n)
    if n == 0
        return 0
    if n == 1
```

```
        return 1
    return FIBONACCI(n-1) + FIBONACCI(n-2)
```

Python Code (Recursive Fibonacci)

```python
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)

n = 5
print(fibonacci(n))
```

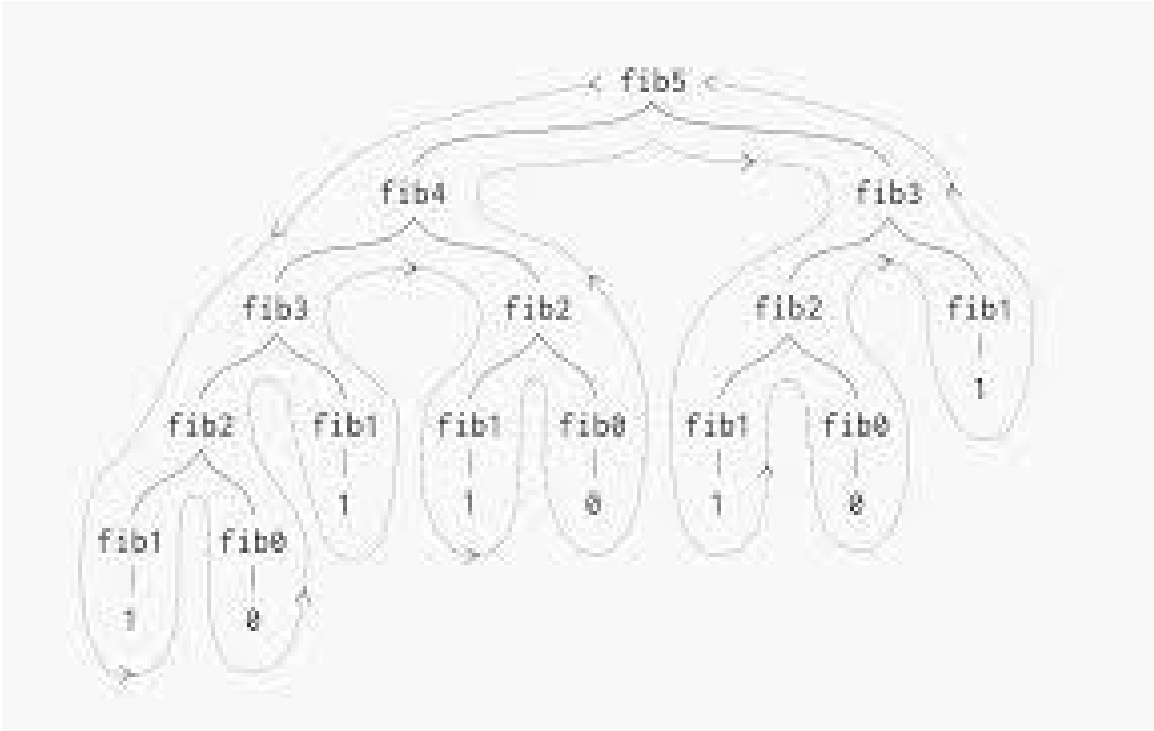
Step-by-Step Tracing

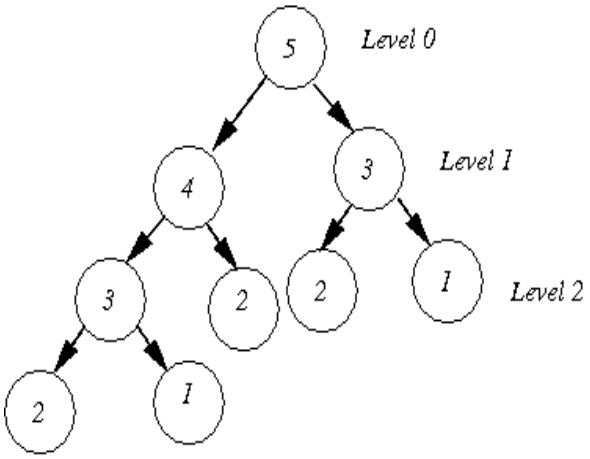
Example: `fibonacci(5)`

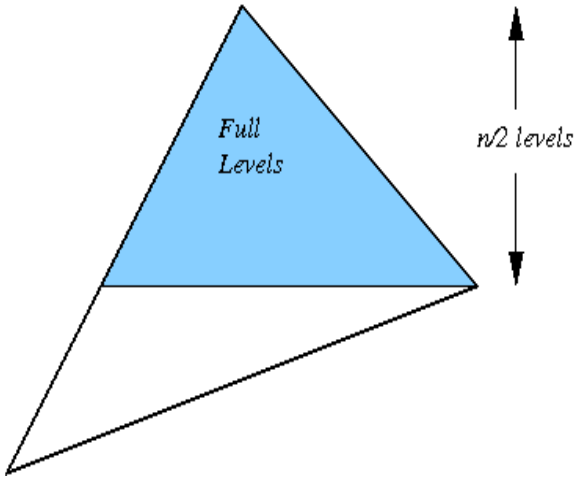
```
fibonacci(5)
 ├─ fibonacci(4)
 │   ├─ fibonacci(3)
 │   │   ├─ fibonacci(2)
 │   │   │   ├─ fibonacci(1) → 1
 │   │   │   └─ fibonacci(0) → 0
 │   │   └─ fibonacci(1) → 1
 │   └─ fibonacci(2)
 │       ├─ fibonacci(1) → 1
 │       └─ fibonacci(0) → 0
 └─ fibonacci(3)
     ├─ fibonacci(2)
     │   ├─ fibonacci(1) → 1
     │   └─ fibonacci(0) → 0
     └─ fibonacci(1) → 1
```

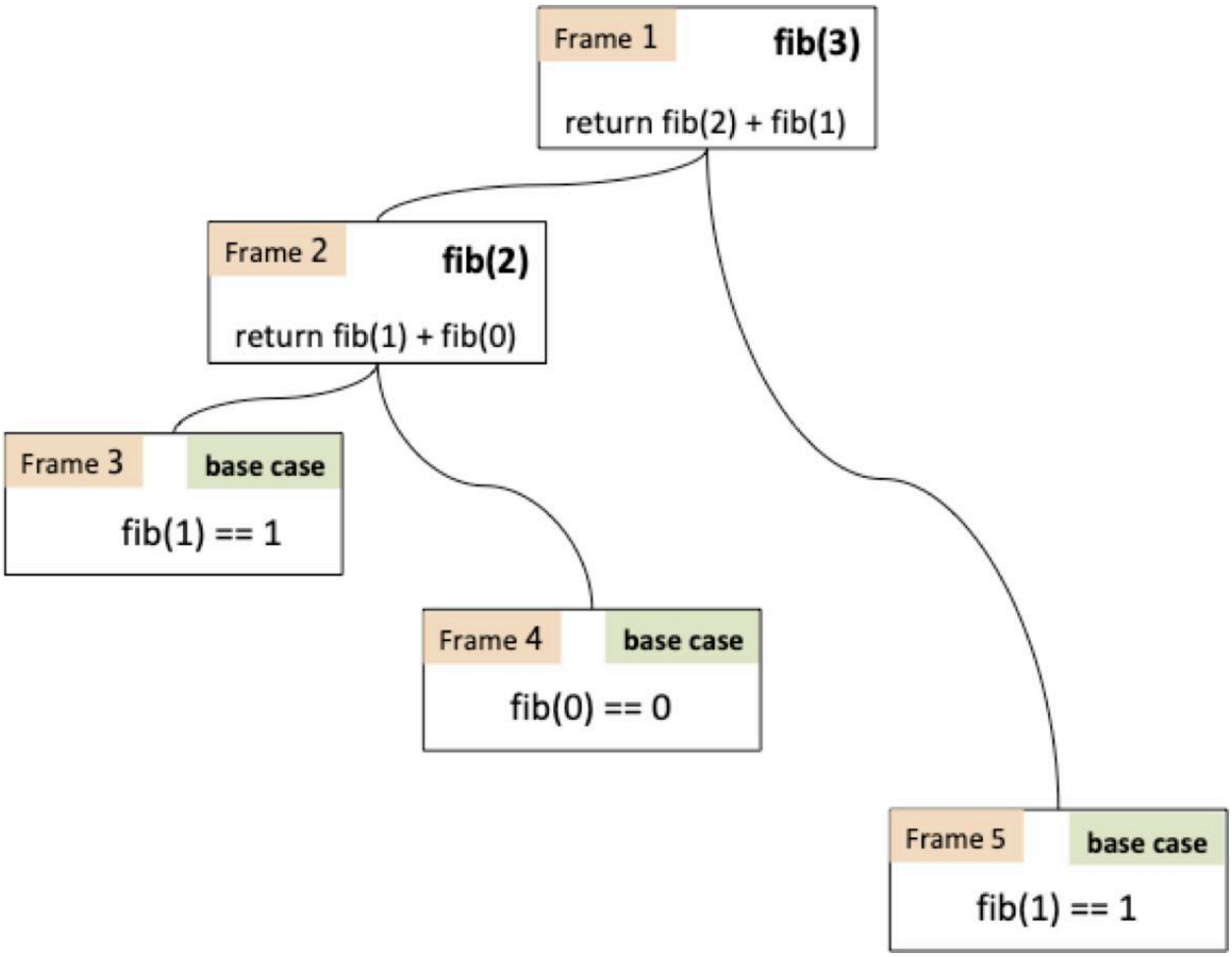
Same values (fibonacci(2), fibonacci(3)) are recomputed many times

Pictographical Representation (Recursion Tree)


fib5
fib4
fib3
fib3
fib2
fib2
fib1
fib2
fib1
fib1
fib0
fib1
fib0
fib1
fib0
1
1
0
1
0
1
0


5
Level 0
4
3
Level 1
3
2
2
1
Level 2
2
1


Full
Levels
n/2 levels



Time Complexity Analysis

- **Time Complexity:**
  O(2n)O(2^n)O(2n)
- **Space Complexity:**
  O(n)O(n)O(n)